

Configurable Cyclic Redundancy Check generator and checker with error detection coverage analysis for PCIe application

¹Ms. MADUGULA POORNIMA, ²Mrs. D RAJITHA, ³Mrs. P SAILAJA

¹PG Scholar, ²ASSOCIATE PROFESSOR, ³ASSISTANT PROFESSOR

Department of Electronics and Communication Engineering

Miracle Educational Society Group of Institutions, Bhogapuram-535216, AP, India

Published online: XX March XXXX

Abstract –In this thesis CRC is applied for PCIe(Pheripheral Component Interconnect Express) transmission in Data Link Layer .Link CRC is implemented and verified using Verilog.In an advanced transmission system, an error occurred when a bit adjusts in the middle of transmission and gathering in a channel. A binary 1 is moved and a binary 0 is getting or a binary 0 is moved and a binary 1 is getting.

Cyclic Redundancy Check is the most important method to detect the errors occurred during transmission of any data. Errors transpired in communication due to internal and external factors i.e. due to components, link, design, noise and interference etc. This work is focused on study of encoder and decoder of CRC

Index Terms – PCIe, Data link layer, Lcrc, crc generator and checker.

I. INTRODUCTION

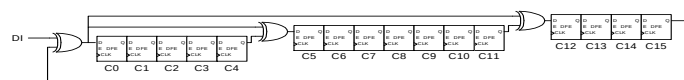
Use of Cyclic Redundancy Codes (CRC) as an alternative to parity and checksum calculations for checking and correcting errors in data transmissions.

The CRC method for error detection and correction treats the data frame as a huge binary number.

The binary number is divided (at the CRC generation end) by a fixed binary number (the CRC generator polynomial) and the resulting remainder of this division (CRC value) is appended to the end of the data frame.

The receiver upon reception of the data frame repeats the calculation and compares its calculated CRC value the CRC value attached to the data frame.

The traditional method for implementing a CRC generator uses a shift register with XOR gates and feedback taps as shown in Figure 1 for the X25 polynomial.



The classic serial implementation is widely used, but it is too slow for PCI Express LCRC and Gigabit Ethernet where bit rates can top 100 Mb/sec.

The alternative method is parallel CRC calculations. This parallel conversion effectively divides the input clock frequency by 8, 16, or 32.

II. RELATED WORK

The use of CRC in PCIe is to identify whether the received data is good or bad. CRC is done both at transmitter and as well as receiver.

At transmitter CRC generator is done where as at the receiver CRC checker is done.

As a generator (TX side)

1. Assert clear high for one cycle to cycle-initialize the module to 0xFFFFFFFF before starting a TLP packet.
2. Assert en and feed your data stream byte-by-byte into data_in.

After the final data byte is evaluated, the crc_out register holds your valid CRC value. Invert the bits (~crc_out) if your link-layer configuration requires final data inversion before serialization

As a checker (RX side)

1. Assert clear high for one cycle before receiving incoming packet data.
2. Stream both the data payload **and** the appended CRC bytes sequentially through data_in while keeping en active.

On the final clock cycle (after the checksum bytes pass through), monitor the `crc_valid` pin. If it goes high, the transaction packet is uncorrupted. If it remains low, a CRC error is detected.

- *Parameters in the PCIe CRC module*
- *Port Interface Signals*
- *Modulo-2 arithmetic*

A. Parameters in the PCIe CRC module [3]

The parameters and constants used in the PCIe CRC code define the mathematical and architectural rules of the **PCI Express specification**.

1. MAGIC_RESIDUE (32'hC704DD7B)

A constant math signature used strictly by the **receiver (checker)** module.

When a transmitting device calculates a CRC, inverts it, and appends it to the end of a data packet, the receiver processes the entire stream (Data + Checksum). Mathematically, if no bits were corrupted during transit, the receiver's internal CRC register will always settle on this precise value, 32'hC704DD7B.

Instead of extracting the checksum and executing a separate comparison block, the hardware simply checks if `(crc_out == MAGIC_RESIDUE)`. This minimizes critical path delays on the RX receiver boundary.

2. CRC_POLY (32'h04C11DB7)

The standard **CRC-32 polynomial** bitmask used by PCIe, Ethernet (802.3), and SATA protocols.

In polynomial division (carried out via binary Modulo-2 arithmetic), this parameter defines the divisor.

The value 32'h04C11DB7 represents the polynomial exponents:

$$\begin{aligned} & \backslash (\mathbf{X}^{\mathbf{32}} + \mathbf{X}^{\mathbf{26}} + \mathbf{X}^{\mathbf{23}} + \mathbf{X}^{\mathbf{22}} + \mathbf{X}^{\mathbf{16}} + \mathbf{X}^{\mathbf{12}} + \mathbf{X}^{\mathbf{11}} + \mathbf{X}^{\mathbf{10}} + \mathbf{X}^{\mathbf{8}} + \mathbf{X}^{\mathbf{7}} + \mathbf{X}^{\mathbf{5}} + \mathbf{X}^{\mathbf{4}} + \mathbf{X}^{\mathbf{2}} + \mathbf{X}^{\mathbf{1}} + \mathbf{1}) \end{aligned}$$

The highest bit (X^{32}) is omitted from the hex parameter because it is implicitly factored into the hardware bit-shifting logic (`next_crc << 1`).

3. 32'hFFFFFFFF (The Initialization Seed)

The starting value loaded into the `crc_out` register when `rst_n` or `clear` is asserted.

Initializing the CRC register to all ones (rather than all zeros) ensures that leading zero-bytes at the start

of a packet payload (e.g., 16'h0000) are mathematically captured and accounted for.

If the register started at 0, leading zeros would not alter the state of the XOR matrix, rendering the circuit blind to dropped or added leading zero-padding.

B. Port Interface Signals [4] Port Interface Signals

Beyond the internal constants, the hardware control pins regulate data flow:

clear: Synchronous register flush. It must be pulsed high for exactly one clock cycle before a new Transaction Layer Packet (TLP) begins to wipe clean the arithmetic remnants of the previous packet.

en (Enable): Flow control flag. If a pipeline stall occurs or a bus stall structure inserts an idle cycle, de-asserting `en` freezes the CRC register state until the next valid 16-bit word arrives.

C. Modulo-2 arithmetic[5]

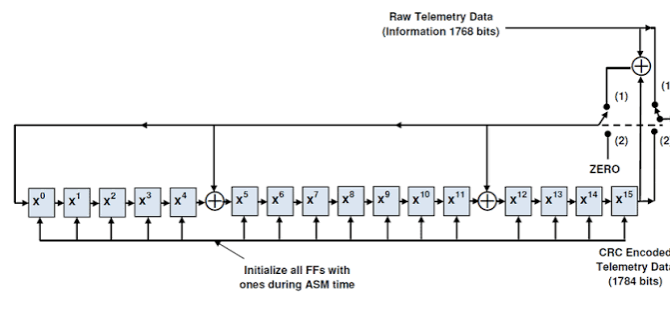


Figure 1

- 1) The algorithm acts on the bits directly above the divisor in each step. The result for that iteration is the bitwise XOR of the polynomial divisor with the bits above it. The bits not above the divisor are simply copied directly below for that step. The divisor is then shifted right to align with the highest remaining 1 bit in the input, and the process is repeated until the divisor reaches the right-hand end of the input row. Here is the entire calculation:

11010011101100 000 <--- input padded by 3 bits from the right

1011 <--- divisor

01100011101100 000 <--- result (the first four bits are the XOR with the divisor

rest of the bits are unchanged) beneath, the

1011 <--- divisor ...

00111011101100 000

1011

00010111101100 000

```

      1011
00000001101100 000 <--- the divisor moves over to
align with the next 1 in the
                        dividend (since quotient for
that step was zero)
      1011          (in other words, it doesn't
necessarily move one bit per iteration)
000000000110100 000
      1011
00000000011000 000
      1011
00000000001110 000
      1011
00000000000101 000
      1011
-----
00000000000000 100 <--- remainder (3 bits). Division
algorithm stops here as

```

dividend is equal to zero.

- The validity of a received message can easily be verified by performing the above calculation again, this time with the check value added instead of zeroes. The remainder should equal zero if there are no detectable errors.

```

11010011101100 100 <--- input with check value
1011                <--- divisor
01100011101100 100 <--- result
1011                <--- divisor ...
00111011101100 100
1011
00010111101100 100
1011
000000001101100 100
1011
-----
000000000110100 100
1011
-----
000000000011000 100
1011
-----
000000000001110 100
1011
-----
000000000000101 100
101 1
-----
000000000000000 000 <----- remainder zero

```

No change in the message signal at the receiver

III. PROPOSED METHOD

This project involves writing synthesizable RTL code (Verilog) for each component. The development process includes:

- Proposed CRC design is done for PCIe by datalink layer when the data transmission takes place between transmitter and receiver.
- Here the receiver checks whether the message data received is correct or not in data link layer by using LCRC (Link CRC).
- It compares with input data, if correct data is received receiver sends ACK status and if error occurs sends NACK status to the transaction layer in the PCIe

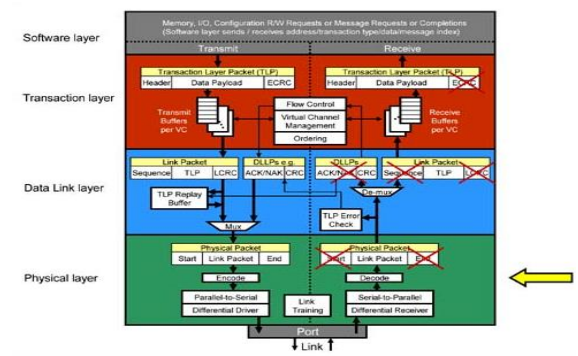


Figure 2

IV. RESULTS AND DISCUSSIONS

RTL level verification is completed and results are shown below:

The data link layer at the transmitter transfers the data generating CRC 32 bits. The data link layer at the receiver checks whether the correct data is received or not.

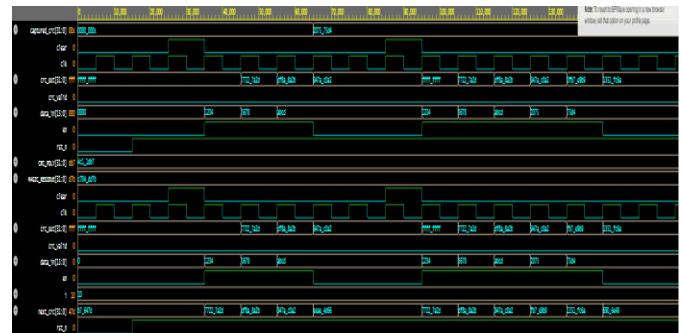


Figure 3: simulated output waveform

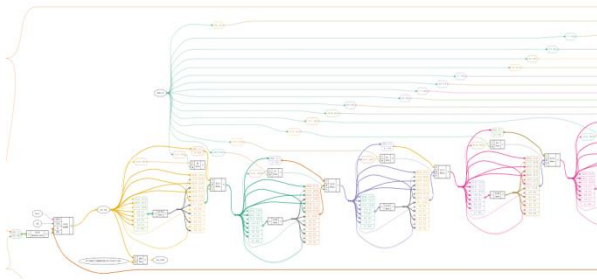


Figure 4:synthesized output

V. CONCLUSION

The data link layer in PCIe at the receiver side checks whether the data is correct or not. If data received is correct it sends ACK acknowledgement to the transportation layer and if wrong data is received it sends NACK.

REFERENCES

- [1] T Stallings, William, "Data and computer communications", Upper Saddle Pearson/Prentice Hall, 8th Edition, 2007. River, N.J.:
- [2] [2] Palani Subbaiah, "Bit- Error Rate for High Speed Serial Data Communication", Data-communications Division, Cypress Semiconductor, November 2008.
- [3] [3] Peterson, W. & E. Weldon, "Error-Correcting Codes", Second Edition, MIT Press, 1972.
- [4] [4] Ulf Nordqvist, Thesis: "Protocol Processing in Network Terminals", Department of Electrical Engineering, Serial Port Complete [by Jan Axelsson

Funding Declaration

The authors declare that no funds, grants, or other forms of financial support were received from any organization or institution for the conduct of this research.